

LOVELY PROFESSIONAL UNIVERSITY

PROJECT REPORT

LPU SMART NAVIGATOR

LPU Pathfinder

A Data Structures and Algorithms Based Campus Navigation System

Submitted By

Dipti Mishra

Registration Number: 12410476

Computer Science and Engineering
Academic Project Documentation
August 2026

DECLARATION

I, Dipti Mishra (Registration Number 12410476), declare that the project titled LPU Smart Navigator / LPU Pathfinder represents my academic project work. This report documents the current design, implementation, algorithms, architecture, testing considerations, limitations, and future scope of the system based on the project source code.

The report distinguishes implemented functionality from proposed future enhancements and does not claim production features that are not present in the implementation.

Student: Dipti Mishra

Registration Number: 12410476

Date: August 2026

ACKNOWLEDGEMENT

The development of LPU Smart Navigator provided an opportunity to connect Data Structures and Algorithms with a practical campus-navigation problem. The project required graph modelling, shortest-path algorithms, geographic data, a C++ backend, and a React-based interactive interface.

The work also provided practical exposure to TypeScript, React, Vite, Leaflet, C++, CMake, HTTP communication, JSON data handling, graph validation, and iterative interface development. This report consolidates the technical work into a structured academic document.

ABSTRACT

LPU Smart Navigator / LPU Pathfinder is a campus navigation system that models the Lovely Professional University environment as a weighted graph and computes walking routes between locations. The project is designed both as a usable campus-navigation interface and as a practical demonstration of Data Structures and Algorithms.

The system uses a React and TypeScript frontend with Leaflet-based map rendering. Campus locations are represented as nodes while walkable connections are weighted edges containing distance, walking time, path type, directionality, and geometry. The C++20 backend contains a graph engine, search index, Dijkstra router, A* router, routing manager, and lightweight HTTP server. Node and edge datasets are stored as JSON.

Dijkstra computes minimum-distance paths using a priority queue, while A* combines accumulated path cost with a Haversine geographic heuristic. RoutingManager can select a primary algorithm and contains a development verification mode that compares Dijkstra and A* results. The frontend sends route requests to the C++ backend and contains a client-side A* fallback when the backend is unavailable.

The project also contains administrative graph editing for nodes and paths, JSON import/export, dataset statistics, and saving graph changes through the backend. The result is an integrated demonstration of graph data structures, pathfinding, API processing, and map visualization.

Keywords: Graph, Dijkstra, A*, shortest path, campus navigation, React, TypeScript, C++, Leaflet, JSON

TABLE OF CONTENTS

Section	Title
1	Introduction
2	Existing System and Technical Background
3	Requirement Analysis
4	System Design and Architecture
5	Data Structures and Algorithms
6	Implementation
7	User Interface and Workflows
8	Testing and Verification
9	Results and Discussion
10	Performance and Complexity
11	Security and Reliability
12	Limitations
13	Future Enhancements
14	Conclusion
A	Project Structure and Setup
B	API Reference
C	Selected Implementation Snippets
R	References

CHAPTER 1

INTRODUCTION

1.1 Background

University campuses contain buildings, gates, hostels, intersections, walkways, parking areas, food courts and other points of interest. A useful navigation system must understand how these locations are connected, not merely display them.

This makes campus navigation a natural graph problem. Locations can be represented as vertices, walking connections as edges, and traversal cost as edge weights. Once this model exists, shortest-path algorithms can calculate routes.

1.2 Problem Statement

The project addresses the need for a campus-specific walking navigation system that demonstrates graph data structures and shortest-path algorithms while providing an interactive map interface.

1.3 Objectives

- Represent campus locations and connections as structured graph data.
- Implement Dijkstra and A* shortest-path routing.
- Expose routing through a C++ HTTP backend.
- Provide an interactive React/TypeScript map interface.
- Allow searchable start and destination selection.
- Display route distance, estimated walking time, and route geometry.
- Provide administrative graph editing and JSON import/export.
- Validate graph consistency and provide graph statistics.

1.4 Scope

The verified scope is campus graph navigation and graph-data management. The system is not presented as a commercial navigation platform and does not currently provide verified live traffic, live GPS, or production-scale security.

1.5 Significance

The academic significance is the direct connection between theory and implementation: graph theory is converted into campus data, algorithms process that data, and the resulting path drives a visual application.

CHAPTER 2

EXISTING SYSTEM AND TECHNICAL BACKGROUND

2.1 Existing Navigation Systems

General-purpose navigation applications provide broad mapping and routing. A campus-specific system can instead maintain a focused dataset of university locations and walking connections.

2.2 Proposed Approach

LPU Smart Navigator combines an interactive map with an explicit weighted graph. The frontend provides interaction and visualization while the C++ backend provides the primary routing computation.

Aspect	General Navigation	LPU Smart Navigator
Focus	Broad geography	Campus graph
Graph data	Usually abstracted	Explicit JSON nodes/edges
Routing	Product-specific	Dijkstra + A*
Graph editing	Usually unavailable	Admin graph editing
Academic DSA role	Secondary	Central

2.3 Graph Theory

A graph $G=(V,E)$ contains vertices V and edges E . In this project, a Node identifies a campus point and an Edge identifies a traversable connection. Edge metadata includes distance, walking time, path type, geometry and bidirectional state.

2.4 Shortest-Path Theory

Dijkstra repeatedly expands the currently smallest known distance. A* ranks nodes using $f(n)=g(n)+h(n)$, where $h(n)$ estimates remaining cost. The implementation uses Haversine geographic distance as its heuristic.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Functional Requirements

ID	Requirement	Status
FR-01	Load nodes and edges from JSON	Implemented
FR-02	Build adjacency list	Implemented
FR-03	Validate graph data	Implemented
FR-04	Search campus locations	Implemented
FR-05	Calculate walking route	Implemented
FR-06	Dijkstra and A* routing	Implemented
FR-07	Leaflet map visualization	Implemented
FR-08	Admin node/path editing	Implemented
FR-09	Import/export graph JSON	Implemented
FR-10	Persistent database	Not implemented

3.2 Non-Functional Requirements

- Usability: searchable location selection and concise route feedback.
- Maintainability: separate graph, routing, HTTP, search, and UI modules.
- Performance: adjacency lists and priority queues support sparse graph traversal.
- Reliability: graph validation and route error handling are built into the implementation.
- Extensibility: graph data can be edited and new node/path types can be represented.

3.3 Verified Technology Stack

Layer	Technology	Purpose
UI	React 19 + TypeScript 6	Application interface
Build	Vite 8	Frontend development/build
Mapping	Leaflet 1.9 + React Leaflet 5	Interactive map
Styling	Tailwind CSS 4	UI styling
Backend	C++20	Graph/routing/API
Build	CMake 3.16+	Backend build
Data	JSON	Node/edge dataset

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

4.1 High-Level Architecture

User/Admin → React + TypeScript → Navigation/API → C++ HTTP Server → RoutingManager → Dijkstra/A* → RouteResult → React/Leaflet map.

4.2 Frontend Modules

Module	Responsibility
App.tsx	Top-level application; renders MapView.
MapView.tsx	Map, nodes, edges, route display, editing, admin workflow, dataset state.
NavigationPanel.tsx	Searchable start/destination selection, navigation action, route summary, modes.
apiClient.ts	HTTP calls for /api/route and /api/save-graph.
aStarRouter.ts	Client-side A* fallback.
dijkstraRouter.ts	Route/result types and frontend routing support.

4.3 Backend Modules

Module	Responsibility
Graph.cpp	Load, validate, index graph data and build adjacency list.
DijkstraRouter.cpp	Dijkstra shortest-path implementation.
AStarRouter.cpp	A* with Haversine heuristic.
RoutingManager.cpp	Algorithm selection, timing and verification.
HttpServer.cpp	Socket HTTP server and API endpoints.
SearchIndex.cpp	Location search indexing.

4.4 Data Model

A node contains an identifier, name, type, latitude, longitude, category and hidden-state flag. An edge contains an identifier, source and destination nodes, geometry, distance, walking time, path type and bidirectional state.

4.5 Data Flow

User selection → NavigationPanel → apiClient → POST /api/route → HttpServer → RoutingManager → selected algorithm → RouteResult → JSON response → map/route display.

CHAPTER 5

DATA STRUCTURES AND ALGORITHMS

5.1 Graph and Adjacency List

Graph.cpp stores nodes in an unordered_map keyed by node ID and edges in a vector. It builds an adjacency list after loading the edge dataset. Bidirectional edges create both forward and reverse entries.

5.2 Dijkstra Algorithm

DijkstraRouter uses a min-priority queue of (distance,node), a distance map, parent map and visited set. When a shorter route to a neighbour is found, its distance and parent are updated.

Representative pseudocode

```
dist[start] = 0
push (0, start) into min-heap
while heap not empty:
    (d,u) = pop minimum
    if u visited: continue
    mark u visited
    if u == destination: stop
    for each neighbour v:
        newDist = d + weight(u,v)
        if newDist improves dist[v]:
            dist[v] = newDist
            parent[v] = (u, edge)
            push (newDist,v)
backtrack parent map
```

5.3 A* Algorithm

AStarRouter maintains gScore values and ranks queue entries by fScore=g+h. Its h function is Haversine distance between the current node and destination. The backend also records the number of expanded nodes.

5.4 Haversine Heuristic

The Haversine formula estimates geographic distance on a sphere. In the backend, the earth radius is 6,371,000 metres. The heuristic is used by A* to prioritize nodes geographically closer to the destination.

5.5 Route Reconstruction

Both routers store predecessor information, backtrack from destination to start, reverse the route, orient edge geometries and merge them into a continuous polyline while removing duplicate adjacent points.

5.6 Complexity

Component	Typical complexity	Role
Graph storage	$O(V+E)$	Adjacency representation
Dijkstra with heap	$O((V+E) \log V)$	Shortest route
A* with heap	$O((V+E) \log V)$ worst-case	Heuristic route
Connectivity BFS	$O(V+E)$	Graph statistics
Node lookup	Average $O(1)$	unordered_map

5.7 Walking Time

The current routers estimate walking time using a fixed speed of 1.4 m/s. This is an estimate, not live GPS timing.

CHAPTER 6

IMPLEMENTATION

6.1 Repository Structure

```
LPU-Smart-Navigator/  
├── frontend/  
│   ├── src/components/  
│   │   ├── MapView.tsx  
│   │   └── NavigationPanel.tsx  
│   ├── src/utils/  
│   │   ├── aStarRouter.ts  
│   │   ├── apiClient.ts  
│   │   └── dijkstraRouter.ts  
│   └── package.json  
├── backend/  
│   ├── data/nodes.json  
│   ├── data/edges.json  
│   ├── include/  
│   ├── src/  
│   │   ├── main.cpp  
│   │   └── CMakeLists.txt  
└── README.md
```

6.2 Frontend

The frontend uses React 19, TypeScript 6, Vite 8, Leaflet 1.9, React Leaflet 5 and Tailwind CSS 4. App.tsx renders MapView, which coordinates the map, graph dataset, editing, route display and NavigationPanel.

6.3 MapView

MapView initializes the map around [31.2536, 75.7037] with an initial zoom of 16. It imports the node dataset from backend/data/nodes.json and renders POIs, navigation nodes, route geometry and admin editing controls.

6.4 NavigationPanel

NavigationPanel provides searchable start and destination controls, validates selection errors, calls the backend, displays route summaries, and uses client-side A* as a fallback if the backend request fails.

6.5 Admin Graph Editing

- Create and edit nodes through map popups.
- Draw and edit path geometry using intermediate waypoints.
- Keep path endpoints locked during geometry editing.
- Insert navigation nodes and use snapping/merge workflows.
- Import/export nodes and edges as JSON.
- Save graph data to the C++ backend.
- Reload the official dataset and inspect graph statistics.

6.6 CMake Configuration

```
cmake_minimum_required(VERSION 3.16)  
project(LPU_Smart_Navigator_Backend VERSION 1.0.0 LANGUAGES CXX)  
set(CMAKE_CXX_STANDARD 20)  
set(CMAKE_CXX_STANDARD_REQUIRED ON)  
set(CMAKE_CXX_EXTENSIONS OFF)
```

CHAPTER 7

USER INTERFACE AND WORKFLOWS

7.1 User Workflow

Step	Action	System behaviour
1	Open application	Map and graph load.
2	Select start	Searchable location list.
3	Select destination	Destination stored.
4	Navigate	POST /api/route.
5	Route calculation	RoutingManager executes algorithm.
6	Result	Distance/time/route geometry returned.
7	Visualization	Route summary and map path shown.

7.2 User vs Admin

Capability	User	Admin
Map view	Yes	Yes
Route navigation	Yes	Yes
Location search	Yes	Yes
Create node	No	Yes
Edit node	No	Yes
Create/edit path	No	Yes
Import/export JSON	No	Yes
Save graph	No	Yes
Dataset statistics	Limited	Yes

7.3 Authentication Note

The frontend contains demonstration admin credentials and explicitly comments that secure backend authentication should replace them for production. The current mechanism is therefore documented as demo-level authentication.

CHAPTER 8

TESTING AND VERIFICATION

8.1 Graph Validation

The graph engine checks duplicate node/edge IDs, missing node references, non-positive distances, invalid geometry and long edges. It also computes connected components and average degree.

8.2 Dijkstra/A* Verification

RoutingManager has a development verification mode that measures both algorithms, compares route existence and total distance, and checks node/edge sequence equality. Equal-cost alternative routes are recognized.

8.3 Test Matrix

ID	Scenario	Expected result	Status
T01	Valid route	found=true route	Implemented
T02	Unknown node	found=false	Implemented
T03	Same source/destination	Zero-distance route	Implemented
T04	Disconnected nodes	found=false	Implemented
T05	Duplicate node ID	Validation error	Implemented
T06	Missing edge endpoint	Validation error	Implemented
T07	Invalid distance	Validation error	Implemented
T08	Backend unavailable	Frontend A* fallback	Implemented
T09	Invalid graph save	HTTP 400	Implemented
T10	Algorithm comparison	Distance/time comparison	Implemented

8.4 Build Commands

```
# Frontend
cd frontend
npm install
npm run dev
```

```
# Backend
cd backend
cmake -B build
cmake --build build
./build/lpu_smart_navigator_backend
```

CHAPTER 9

RESULTS AND DISCUSSION

9.1 End-to-End Result

The repository implements an end-to-end navigation flow: structured campus data is loaded, the user selects locations, the frontend sends a request, the C++ routing layer computes a path, and the result is returned with distance, walking time, node/edge sequences and map geometry.

9.2 Algorithmic Result

Dijkstra and A* are independent router classes managed through RoutingManager. This makes the project suitable for comparing classic shortest-path search with heuristic-guided search.

9.3 Data Management Result

The project uses JSON rather than a database. POST /api/save-graph validates node references, writes nodes.json and edges.json, and hot-reloads the graph and search index.

9.4 Visualization Result

Leaflet and React Leaflet render campus locations and route geometry. The navigation panel shows the start/destination pair, distance, estimated walking time and route-node count.

9.5 Overall Discussion

The key result is integration: the algorithms form the computational core of a complete application rather than remaining isolated examples. The same graph supports routing, search, validation, editing and visualization.

CHAPTER 10

PERFORMANCE AND COMPLEXITY

10.1 Dijkstra

With an adjacency list and heap-based priority queue, Dijkstra has the conventional complexity $O((V+E) \log V)$. Graph storage is $O(V+E)$.

10.2 A*

A* has the same broad heap-based worst-case family, $O((V+E) \log V)$, while practical search effort can be lower when the heuristic effectively guides expansion. The implementation records nodes expanded.

10.3 Runtime Metrics

Metric	Meaning
Distance	Sum of selected edge distances in metres.
Walking time	Distance / 1.4 m/s.
Nodes expanded	Tracked by A*
Execution time	Measured in milliseconds.
Connected components	BFS-based graph connectivity.
Average degree	Average adjacency degree.

10.4 Scalability

The adjacency-list model is suitable for a sparse campus graph. Larger deployments would require stronger persistence, connection management, caching, automated tests and production server hardening.

CHAPTER 11

SECURITY AND RELIABILITY

11.1 Current Security State

The current system is an academic/local implementation. Admin credentials are present in the frontend as demonstration authentication, and the graph-save API does not provide production-grade authorization.

11.2 CORS

The backend emits permissive CORS headers for local frontend-backend communication. In a production system, allowed origins should be restricted.

11.3 Reliability Mechanisms

- Graph validation catches structural errors.
- Routing handles missing nodes and identical source/destination.
- Frontend displays route-selection and backend errors.
- Frontend A* fallback supports backend outage scenarios.
- Graph-save failures return explicit JSON errors and HTTP 400 responses.

11.4 Recommended Production Security

- Backend authentication and role-based authorization.
- No plaintext credentials in frontend source.
- Restricted CORS.
- Payload validation and sanitization.
- HTTPS outside local development.
- Audit logging and rate limiting for administrative changes.

CHAPTER 12

LIMITATIONS

12.1 Current Limitations

- JSON-file graph persistence rather than a dedicated database.
- Lightweight custom socket HTTP server.
- Demo-level admin authentication.
- Curated campus graph rather than live traffic data.
- Fixed walking-speed estimate.
- No verified real-time GPS tracking.
- No production load/concurrency benchmark in the repository.

12.2 Interpretation

These limitations are consistent with an academic project whose central objective is demonstrating DSA inside a working software system. They also define clear future engineering tasks.

CHAPTER 13

FUTURE ENHANCEMENTS

13.1 Data and Persistence

- Database-backed nodes, edges, users and change history.
- Graph versioning and rollback.
- Controlled data-import and validation pipeline.

13.2 Navigation

- Alternative routes.
- Accessibility-aware routing.
- Indoor/floor-aware navigation.
- Dynamic weights using real-time conditions.

13.3 Smart Features

- Crowd-aware routing.
- Event-aware route restrictions.
- Personalized routing.
- Voice-assisted navigation.

13.4 Production Engineering

- Secure backend authentication.
- Production HTTP architecture.
- Automated tests and CI.
- Large-graph performance benchmarking.
- Secure configuration and secrets management.

CHAPTER 14

CONCLUSION

LPU Smart Navigator / LPU Pathfinder demonstrates how a real campus navigation problem can be transformed into a graph problem and then into a complete software solution. Campus locations are represented as nodes, walking connections as weighted edges, and route computation is handled by Dijkstra and A*.

The C++20 backend provides the graph and routing engine, while the React/TypeScript frontend provides the interactive map and navigation controls. Graph validation, search, administrative editing, JSON import/export, backend persistence and development-time algorithm comparison extend the project beyond a basic shortest-path demonstration.

The project therefore demonstrates the progression from problem modelling to data structures, algorithms, API communication, user interaction and visualization. Its current limitations provide meaningful directions for future work toward a more secure, scalable and persistent navigation platform.

Project narrative: Real-world campus problem → weighted graph → DSA → Dijkstra/A* → C++ routing backend → React/TypeScript interface → interactive campus navigation.

CHAPTER A

PROJECT STRUCTURE AND SETUP

A.1 Repository Structure

```
LPU-Smart-Navigator/  
├── frontend/  
│   ├── src/components/  
│   │   ├── MapView.tsx  
│   │   └── NavigationPanel.tsx  
│   ├── src/utlis/  
│   │   ├── aStarRouter.ts  
│   │   ├── apiClient.ts  
│   │   └── dijkstraRouter.ts  
│   └── package.json  
├── backend/  
│   ├── data/nodes.json  
│   ├── data/edges.json  
│   ├── include/  
│   ├── src/  
│   ├── main.cpp  
│   ├── CMakeLists.txt  
└── README.md
```

A.2 Frontend Setup

```
cd frontend  
npm install  
npm run dev
```

A.3 Backend Setup

```
cd backend  
cmake -B build  
cmake --build build  
./build/lpu_smart_navigator_backend
```

A.4 Verified Dependencies

Package	Version	Purpose
react	19.2.8	UI
react-dom	19.2.8	DOM rendering
leaflet	1.9.4	Map
react-leaflet	5.0.0	React map integration
typescript	~6.0.2	Typing
vite	8.2.0	Build/dev
tailwindcss	4.3.3	Styling
oxlint	1.75.0	Linting

CHAPTER B

API REFERENCE

B.1 POST /api/route

The frontend sends startNodeId and destinationNodeId to `http://localhost:8080/api/route`. The backend passes the request to `RoutingManager` and serializes a `RouteResult`.

Field	Type	Meaning
found	boolean	Route existence
totalDistance	number	Metres
walkingTime	number	Estimated walking time
nodeIds	string[]	Ordered route nodes
edgeIds	string[]	Ordered route edges
geometry	number[][]	Latitude/longitude polyline

B.2 POST /api/save-graph

The endpoint accepts nodes and edges arrays, validates edge references, writes `data/nodes.json` and `data/edges.json`, then hot-reloads the graph and search index.

Condition	Response
Missing arrays	HTTP 400
Missing node reference	HTTP 400
Valid graph	HTTP 200 with success and counts
Parsing/write failure	HTTP 400

B.3 OPTIONS

OPTIONS is handled for CORS preflight.

CHAPTER C

SELECTED IMPLEMENTATION SNIPPETS

C.1 Dijkstra Priority Queue

```
using Element = std::pair<double, std::string>;
std::priority_queue<
    Element,
    std::vector<Element>,
    std::greater<Element>
> pq;
```

The priority queue selects the currently smallest distance and drives the backend Dijkstra traversal.

C.2 Frontend Route Request

```
fetch(`${BACKEND_BASE_URL}/api/route`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ startNodeId, destinationNodeId }),
})
```

This connects the navigation panel to the C++ route service.

C.3 Bidirectional Edge Handling

```
adjList[edge.fromNodeId].push_back(forwardEdge);
if (edge.isBidirectional) {
    adjList[edge.toNodeId].push_back(reverseEdge);
}
```

This preserves one-way versus bidirectional traversal.

CHAPTER R

REFERENCES

[1] LPU Smart Navigator / LPU Pathfinder project repository, diptim06/LPU-Smart-Navigator, GitHub. Accessed August 2026.

[2] React Documentation, React official documentation.

[3] TypeScript Documentation, TypeScript official documentation.

[4] Leaflet Documentation, Leaflet official documentation.

[5] React Leaflet Documentation, React Leaflet documentation.

[6] OpenStreetMap Documentation, OpenStreetMap project documentation.

[7] CMake Documentation, CMake official documentation.

[8] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C., Introduction to Algorithms, MIT Press.

Primary technical basis: the current public repository was inspected for its frontend structure, backend source files, CMake configuration, routing algorithms, API implementation and graph data.